

I. INTRODUCTION

This notebook runs through my process of extracting data from the spotify API and using it to predict the genre of a given song. As for many data science projects, most of the time was spent figuring out how to obtain and organize the data so that it can be used in a prediction model. The process consisted of two main steps: Compiling song data (obtaining 1000 songs per genre and converting them into feature vectors) and building the model (adjust data, feed into model, evaluate model).

Originally, this project was going to predict which of my personal genre playlists a song should fit into. However, when I actually made a model based off of these specific playlists, I noticed that the predictions were significantly better for songs I was familiar with as opposed to songs that were from completely unfamiliar artists and very new to me. Since each playlist that I have made is tuned to my taste and biased towards certain artists, then it would make sense that predictions are way better for songs that I personally liked and were more closely related to the songs in my playlists. If I had a completely new and unfamiliar song that I discovered, the prediction would not be as great. I wanted to make a less biased model for genre prediction, so I used the genre playlists curated by Spotify to get more diverse music into my training dataset.

In this project, I will use Spotify's API to extract audio features such as danceability and popularity and use these to try and classify songs into genres with different machine learning models. Then I will see which models performed better or worse.

II. COMPILING SONG DATA

To begin the process, first song data must be compiled into a Pandas dataframe (a table in Python that is easy to work with). The playlists selected only have about 50 songs, but for more refined training data, more songs will be generated so that each playlist has about 500 songs. Then each song in each playlist will be iterated over, and the song name, audio features, and playlist genre will be added to the dataframe.

First the necessary libraries will be imported and then a token is generated from <https://developer.spotify.com/console/get-audio-features-track/?id=> and saved. This token will be passed into each request to grant access to various spotify services. This token expires about every 2 hours so a new one would need to be generated. The token was granted all scopes because this project requires creating and manipulating new playlists on a spotify account.

```
import json
import requests
import pandas
import numpy
import csv
user_id = "good."
token = "BQCLdAYixvD8G50z-
oIS1DojsJ_FtX0bWwZS_cKv8CYtpkJuQrjnKYlbmFltW3MZnNVAf0VZRjDsxp_OctQ35d
M8ogj3sJLzLg6ualD4mb6rzkPHhAlkKwvCOYK0XaegjTnPD-
tYS7KwKp6jwxszDyZ3ZWjVhEDuEG6CjD0XAR_zxAtF-iwG6yLVPM_f1E2i03LKw-
```

```
fUWiiWEisu4Z_U0KTcnhpF78D95EAn0FMdkCej0YMzVUL42J8X2AXf098K1s7D23QgC90S3_WbQ"
```

Collect the URI's for each song in each playlist

The playlists used for training each genre were: Rap Caviar (hip-hop), Songs We Rocked Out To (rock), Lowkey (mellow), and Discoland (dance). Since Spotify's playlists cannot be edited to be longer, we will create a copy of each playlist. First the URI of every song will be collected and added to a 2d array (grouped by each playlist).

```
playlists = ["37i9dQZF1DWT3pfwRgyQMA", "37i9dQZF1DX2yvm10dMYzV",
"37i9dQZF1DX9ZABYPehUEa", "37i9dQZF1DX0XUsuxWHRQd"] # rock, chill,
party, rap
playlist2d = [] # A 2d array that will hold all the songs in the
training set, grouped by playlist genre.
for playlist_id in playlists: # Collect the URI for each song and add
it to playlist 2d
    playlist1=[]
    # This query gets a list of all the songs in the playlist
    query =
"https://api.spotify.com/v1/playlists/{}/tracks".format(playlist_id)
    response = requests.get (query, headers={
                                "Content-Type": "application/json",
                                "Authorization": "Bearer
{}".format(token)
                            })

    data = response.json()
    items = data["items"]
    # The URI of each song in the playlist is added to a temporary
playlist (playlist1) which is then added to playlist2d
    for index in range(0,len(data["items"])):
        playlist1.append(items[index]['track']['uri'])
    playlist2d.append(playlist1)
```

Create a working copy of the spotify playlist

Now we can create the copies. A new playlist is created and populated with the songs from each of the playlists, using the URI's in the 2d playlist

```
count = 1
newlists = [] # Keeps track of the identifiers for the copies of the
playlists
print("Adding songs...")
for playlist in playlist2d:
    query =
"https://api.spotify.com/v1/users/{}/playlists".format(user_id)
    # Create a new playlist
```



```

{}".format(token)
    })
    song_list=response.json()["tracks"]
    for song in range(0, 10): # add 10 recommended songs to the
playlist
        if len(song_list)>0:
            query =
"https://api.spotify.com/v1/playlists/{}/tracks?
uris={} ".format(newlists[count],song_list[song]["uri"])
            response = requests.post (query, headers={
                "Content-Type":
"application/json",
                "Authorization": "Bearer
{} ".format(token)
            })
print("Complete")

```

Extracting features from a song

Now that all the songs that we want to train on have been collected, we will create a function that will extract the right data from each song and store it as an array of data values. Each of these data values for each song will eventually end up in a huge spreadsheet that contains the URI for every single song from every playlist, their associated audio feature values, and the playlist the song is assigned to.

```

def features(song_id, label, token): # given a song and assigned
playlist, creates a row of feature data.
    song = []

    # general track details [popularity]
    query = "https://api.spotify.com/v1/tracks?ids={} ".format(song_id)
    response = requests.get (query, headers={
        "Content-Type": "application/json",
        "Authorization": "Bearer
{} ".format(token)
    })

    data=response.json()
    song.append(data["tracks"][0]["popularity"])

    # audio features [length, danceability, acousticness, energy,
instrumentalness, valence, loudness, speechiness, tempo, liveness]
    query = "https://api.spotify.com/v1/audio-
features/{} ".format(song_id)
    response = requests.get (query, headers={
        "Content-Type": "application/json",
        "Authorization": "Bearer
{} ".format(token)
    })

    data = response.json()

```

```

    #song.extend([data["duration_ms"],
data["danceability"],data["acousticness"],data["energy"],data["instrumentalness"],data["valence"],data["loudness"],data["speechiness"],
data["tempo"],data["liveness"],label])

song.extend([data["danceability"],data["acousticness"],data["energy"],
data["valence"],data["loudness"], data["tempo"],label])
return(song)

features('7ouMYWpwJ422jRcDASZB7P',1, token) # Example of what the
output should produce

```

Create a CSV

Using the function we created, we can now pass each song through the features function and obtain an array of data values for it, which we can write to a CSV file. By the end of the following code block, we will have a new CSV file called data.csv which will contain all of our song data.

```

#training_playlists =
['4pBKA5oW0ka1vVfRTeiPXV','24MSqGc7PFwaHU3BlXZN5s','3VshJiKPkMfV50uTk6
jV98','3MN8AsCs9aIzMYTs9rLIa7','3BqN6C25NxEmJcYkjC5fLy']

count = 0
f = open('data.csv', 'w', encoding='UTF8', newline='')
header = ['popularity', 'length', 'danceability',
'acousticness','energy','instrumentalness','valence','loudness','speechiness','tempo','liveness','playlist']
writer = csv.writer(f)
writer.writerow(header)

for id in newlists:
    print("Adding playlist",id," to CSV.")
    count=count+1
    #get songs in list
    setof100=0
    for i in range(0,11):
        query = "https://api.spotify.com/v1/playlists/{}/tracks?
offset={}&limit=100".format(id,i)
        response = requests.get (query, headers={
            "Content-Type":
"application/json",
            "Authorization": "Bearer
{}".format(token)
        })
        data = response.json()
        # iterate through all the songs and get the song id.
        for songnum in range(0,len(data['items'])):
            row=[]

```

```

song_id = data['items'][songnum]['track']['id']
if (count==1):
    row = features(song_id, 'rock', token)
elif(count==2):
    row = features(song_id, 'mellow', token)
elif(count==3):
    row = features(song_id, 'party', token)
else:
    row = features(song_id, 'rap', token)
writer.writerow(row)

f.close()

```

III. EXPLORATORY ANALYSIS

So far, we have a CSV file with all of the song data that we can use to try and predict genres. To get a better idea of the data we have, we can create some charts to visualize how genre might affect the distribution of various audio features. This can help us understanding how Spotify's features interact with actual song data and which features might be the most useful to include when building our machine learning models.

Features

In order to make sense of the graphs, we should first clarify what each metric measured by Spotify actually represents. Here is a brief rundown of each metric as per Spotify's API reference:

acousticness: A confidence measure from 0.0 to 1.0 of whether the track is acoustic. 1.0 represents high confidence the track is acoustic.

danceability: Describes how suitable a track is for dancing based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity. 1 is the most danceable.

energy: A measure from 0.0 to 1.0 and represents a perceptual measure of intensity and activity. Typically, energetic tracks feel fast, loud, and noisy. For example, death metal has high energy, while a Bach prelude scores low on the scale. Perceptual features contributing to this attribute include dynamic range, perceived loudness, timbre, onset rate, and general entropy.

instrumentalness: "Ooh" and "aah" sounds are treated as instrumental in this context. Rap or spoken word tracks are clearly "vocal". The closer the instrumentalness value is to 1.0, the greater likelihood the track contains no vocal content. Values above 0.5 are intended to represent instrumental tracks, but confidence is higher as the value approaches 1.0.

loudness: The overall loudness of a track in decibels (dB). Loudness values are averaged across the entire track and are useful for comparing relative loudness of tracks. Loudness is the quality of a sound that is the primary psychological correlate of physical strength (amplitude). Values typically range between -60 and 0 db.

speechiness: Speechiness detects the presence of spoken words in a track. The more exclusively speech-like the recording (e.g. talk show, audio book, poetry), the closer to 1.0 the attribute value. Values above 0.66 describe tracks that are probably made entirely of spoken words.

tempo: The overall estimated tempo of a track in beats per minute (BPM). In musical terminology, tempo is the speed or pace of a given piece and derives directly from the average beat duration.

valence: A measure from 0.0 to 1.0 describing the musical positiveness conveyed by a track. Tracks with high valence sound more positive (e.g. happy, cheerful, euphoric), while tracks with low valence sound more negative (e.g. sad, depressed, angry).

The first step would be to import all the necessary modules for graphing. This example will use matplotlib. After importing matplotlib, we will create a dataframe for each genre so that graphing by genre is a simpler process.

```
import matplotlib.pyplot as plt
import numpy as np

# Create a dataframe for each genre, and then remove the header
df = pandas.read_csv('data.csv')
df_rock = df[df['playlist'] == 'rock'].iloc[:, :-1]
df_mellow = df[df['playlist'] == 'mellow'].iloc[:, :-1]
df_party = df[df['playlist'] == 'party'].iloc[:, :-1]
df_rap = df[df['playlist'] == 'rap'].iloc[:, :-1]
```

Once the dataframes are set up for each genre, we can go through the list of metrics, and for each metric we can create a plot. The plot will have a line representing each of the four genres, revealing their distribution.

```
# Creating the plots
fig, ax = plt.subplots()

metrics = df_rock.columns # Get a list of metrics based on what the
                           # headers of the columns are

# Create a graph that displays the distribution of values for each
# metric based on genre
for metric in metrics:
    y, binEdges = np.histogram(df_rock[metric], bins=15) # Plot Rock
    bincenters = 0.5 * (binEdges[1:] + binEdges[:-1])
    plt.plot(bincenters, y, '-', c='blue', label='rock')

    y, binEdges = np.histogram(df_party[metric], bins=15) # Plot Party
    bincenters = 0.5 * (binEdges[1:] + binEdges[:-1])
    plt.plot(bincenters, y, '-', c='salmon', label='party')

    y, binEdges = np.histogram(df_mellow[metric], bins=15) # Plot
Mellow
    bincenters = 0.5 * (binEdges[1:] + binEdges[:-1])
    plt.plot(bincenters, y, '-', c='purple', label='mellow')

    y, binEdges = np.histogram(df_rap[metric], bins=15) # Plot Rap
    bincenters = 0.5 * (binEdges[1:] + binEdges[:-1])
```

```

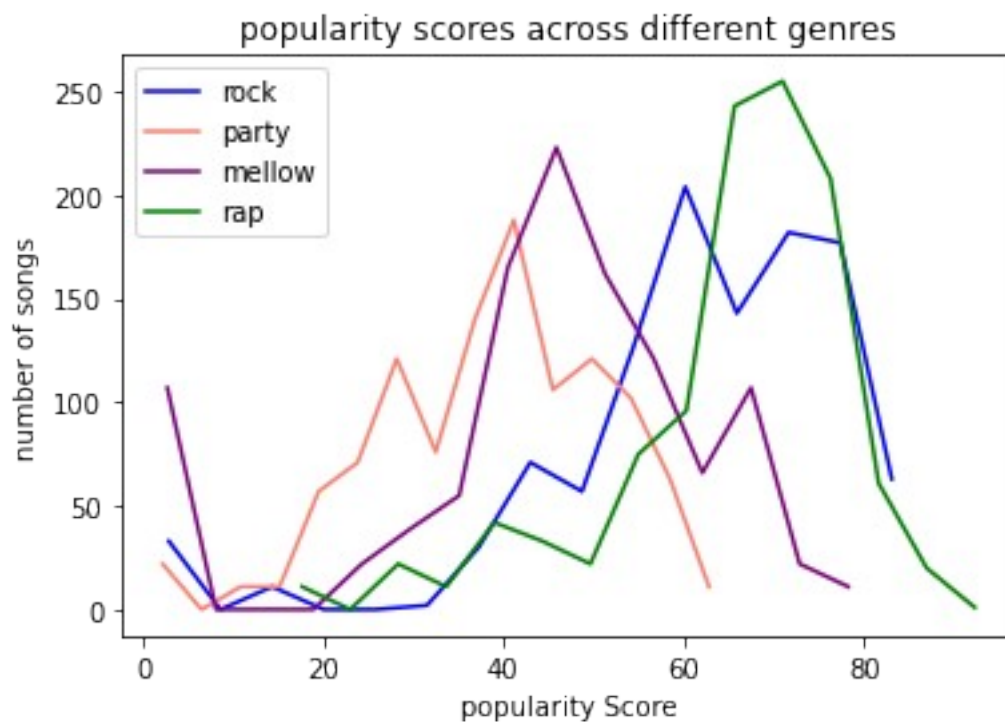
plt.plot(bincenters, y, '--', c='green',label='rap')
leg = ax.legend();

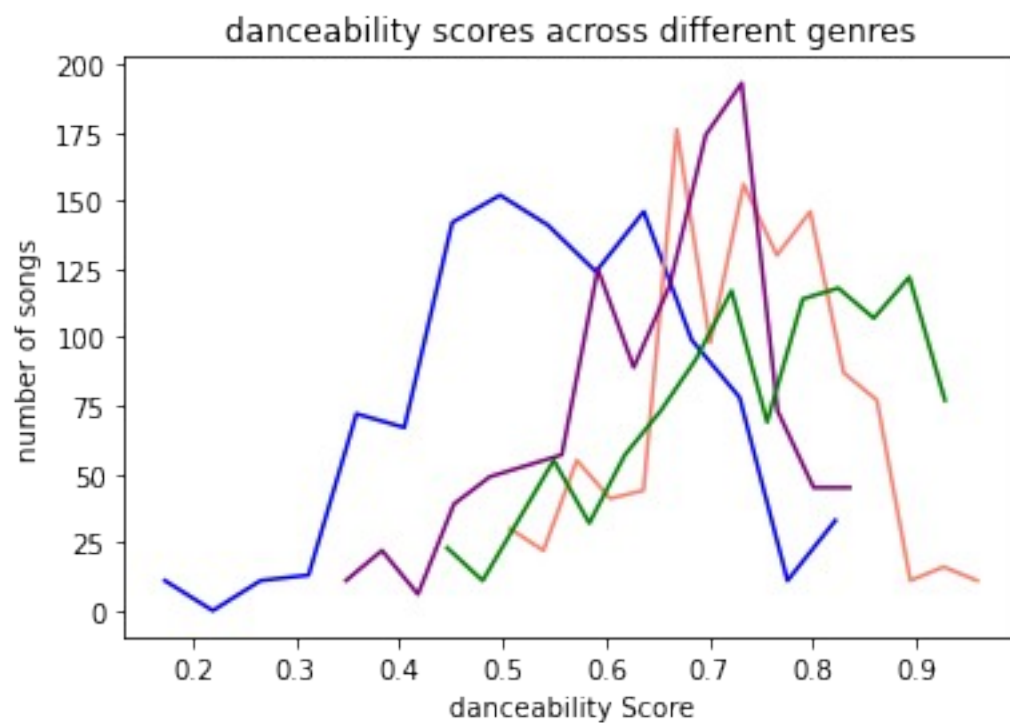
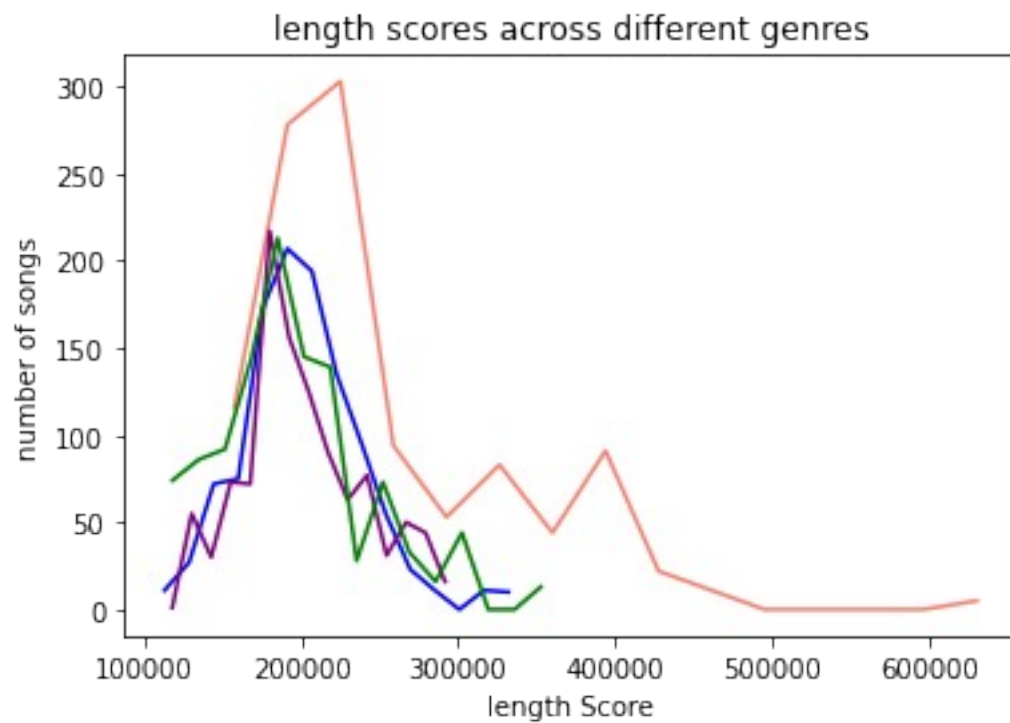
#x and y-axis labels
plt.xlabel(metric+' Score') # Set labels
plt.ylabel('number of songs')

#plot title
plt.title(metric+' scores across different genres')

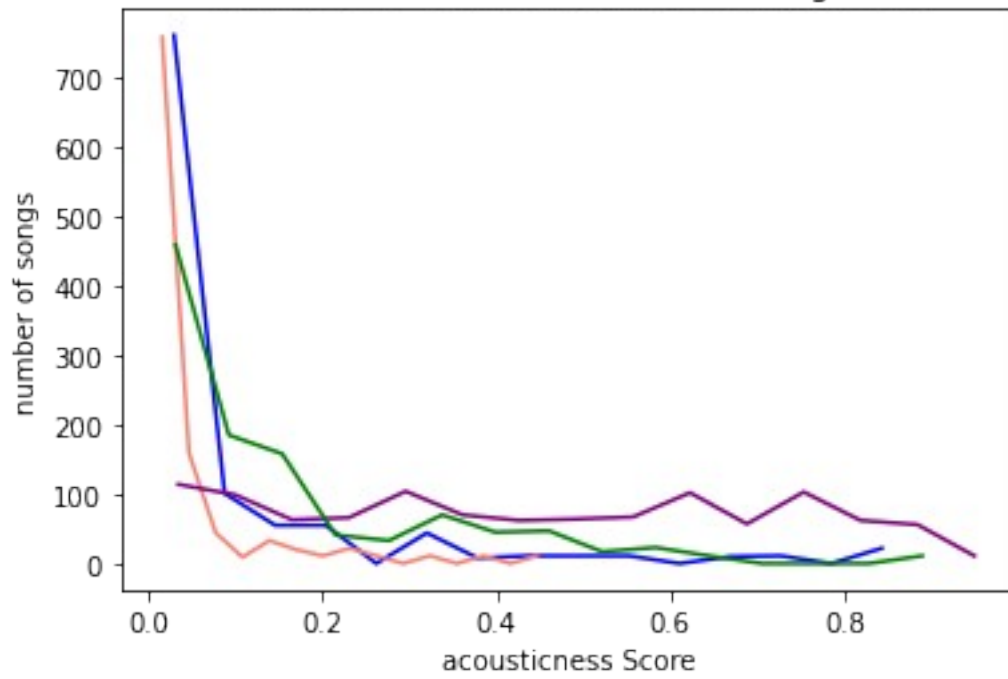
#save and display the plot
plt.show();

```

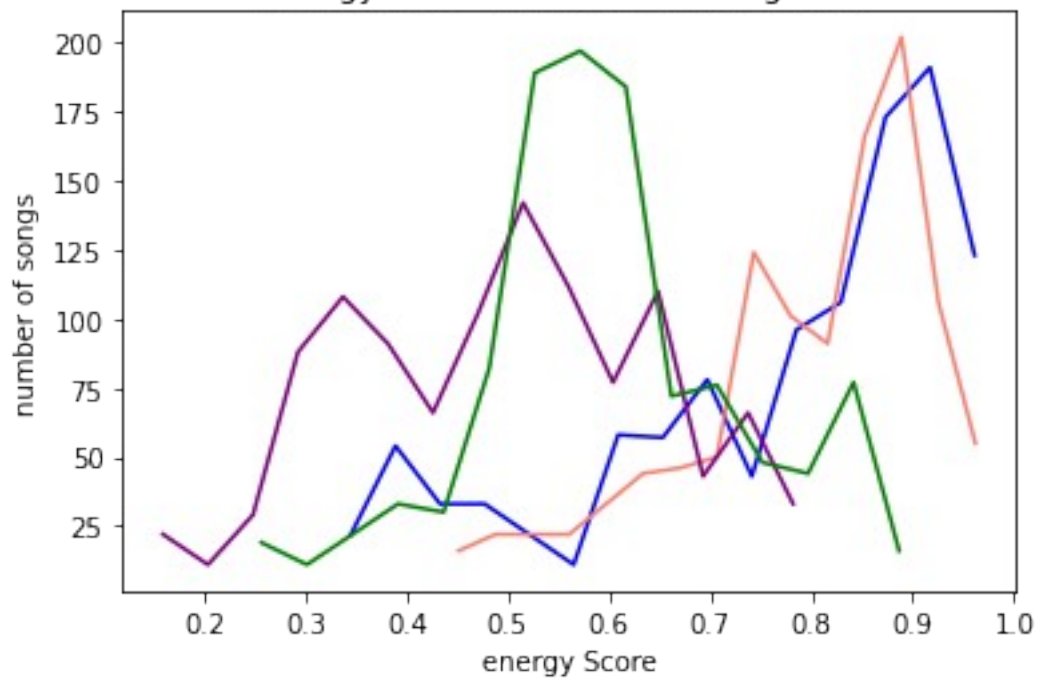




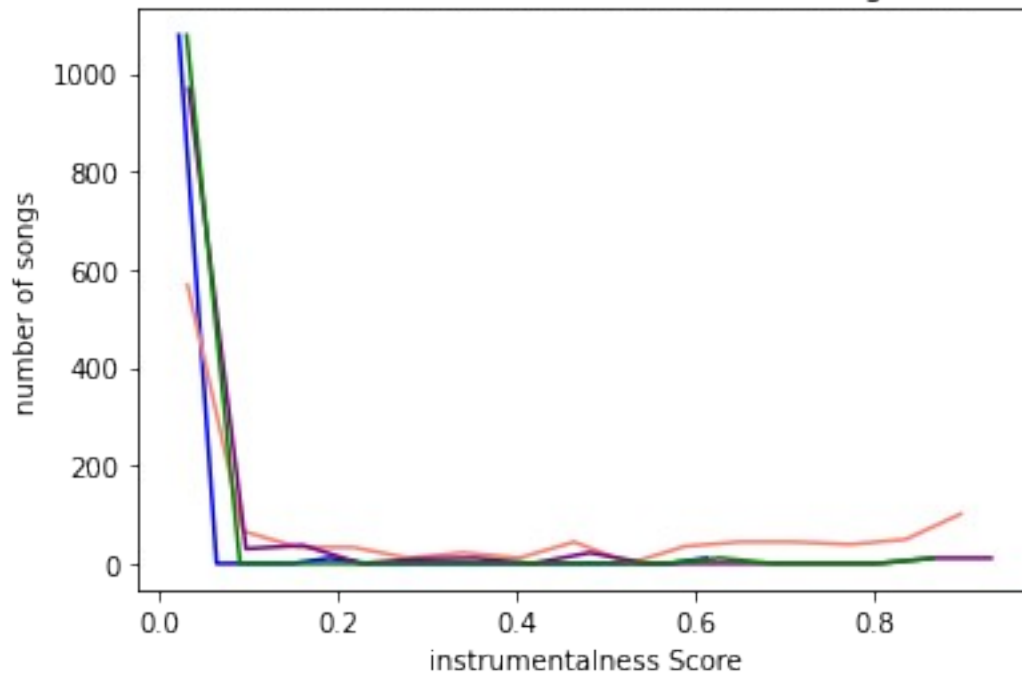
acousticness scores across different genres



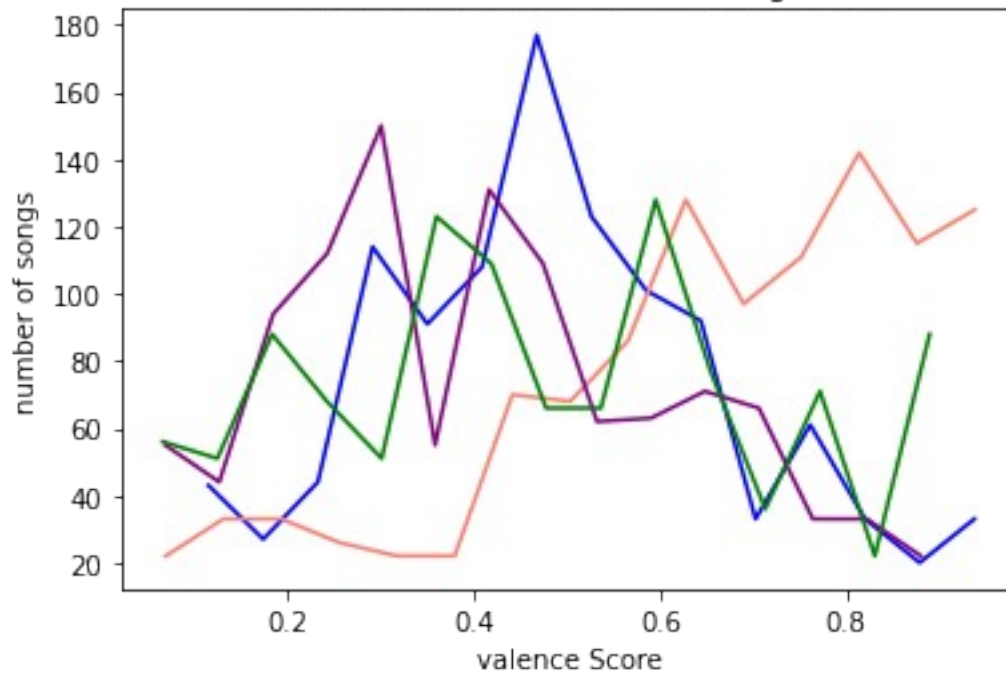
energy scores across different genres

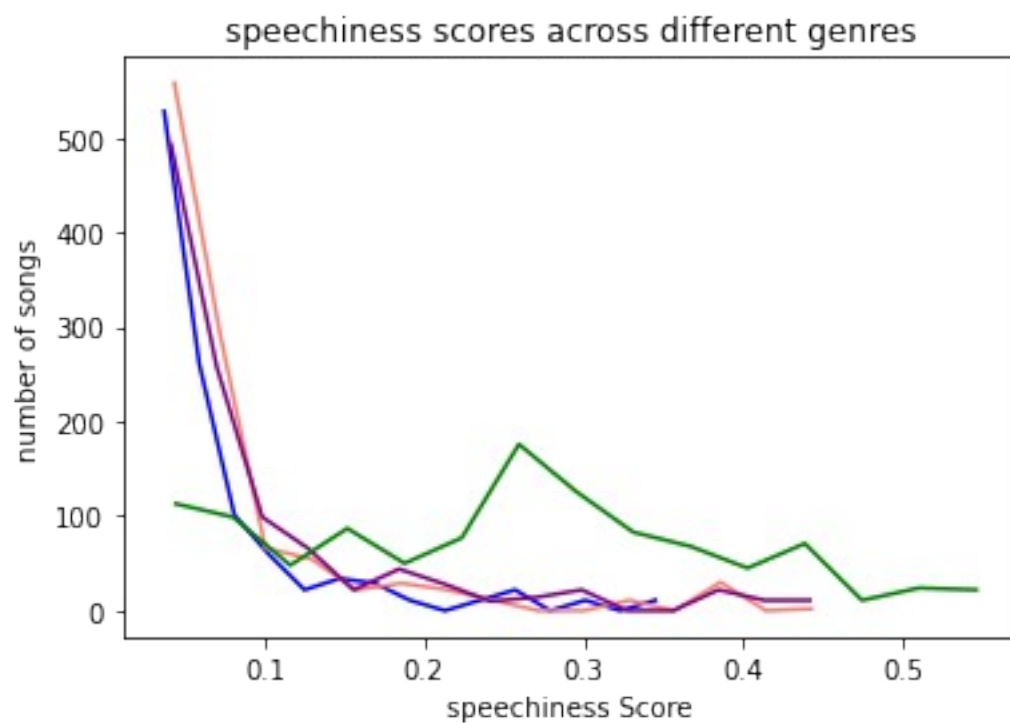
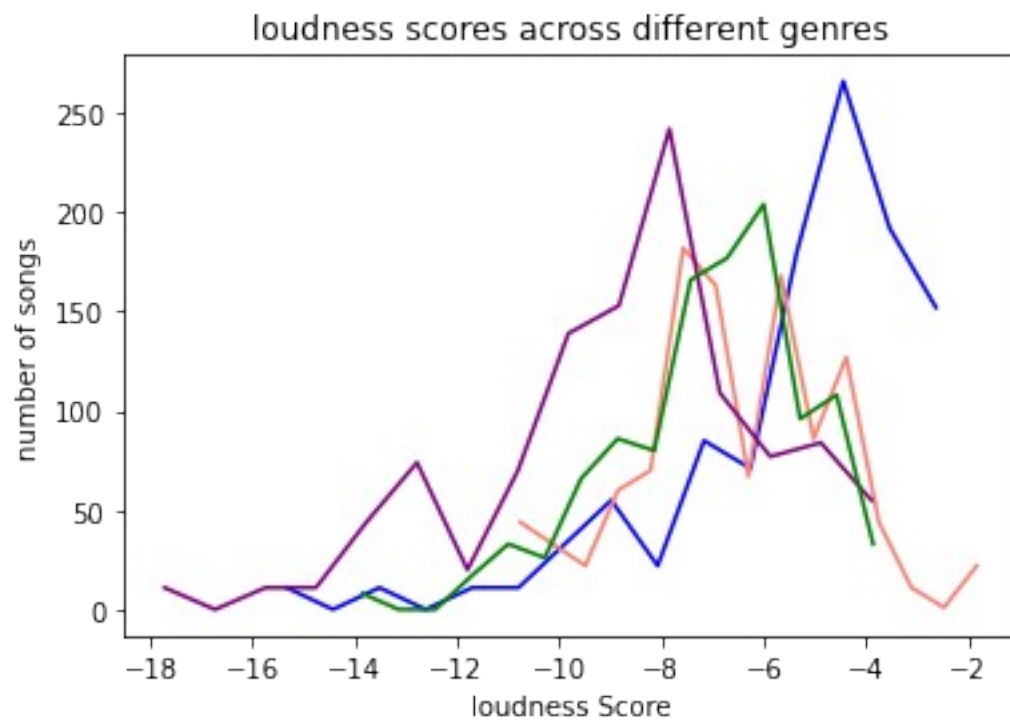


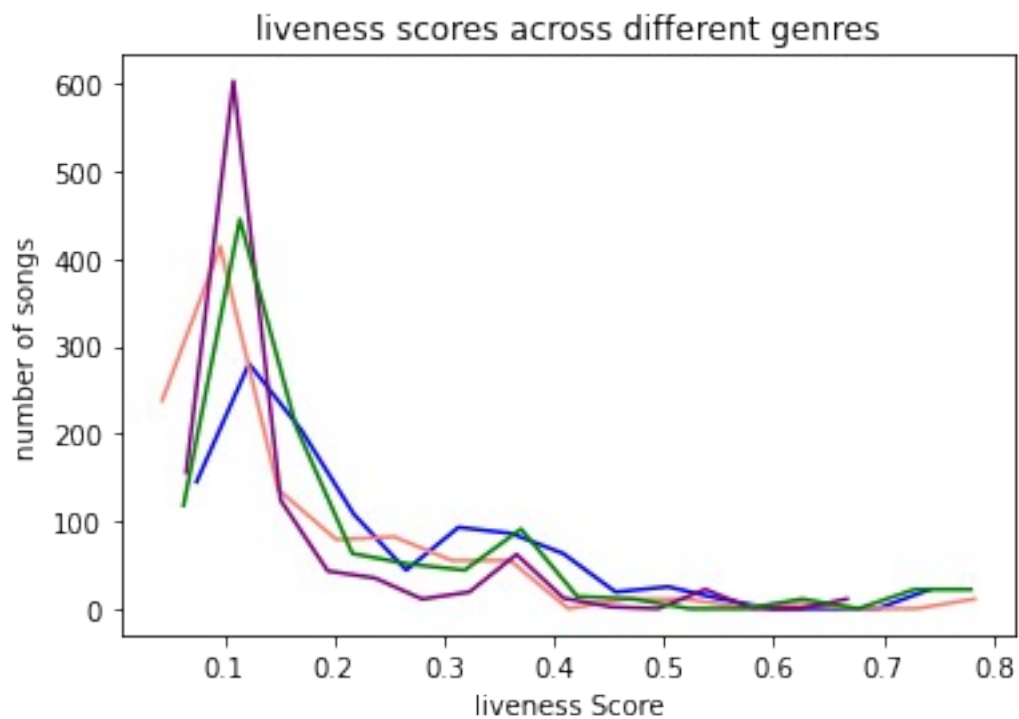
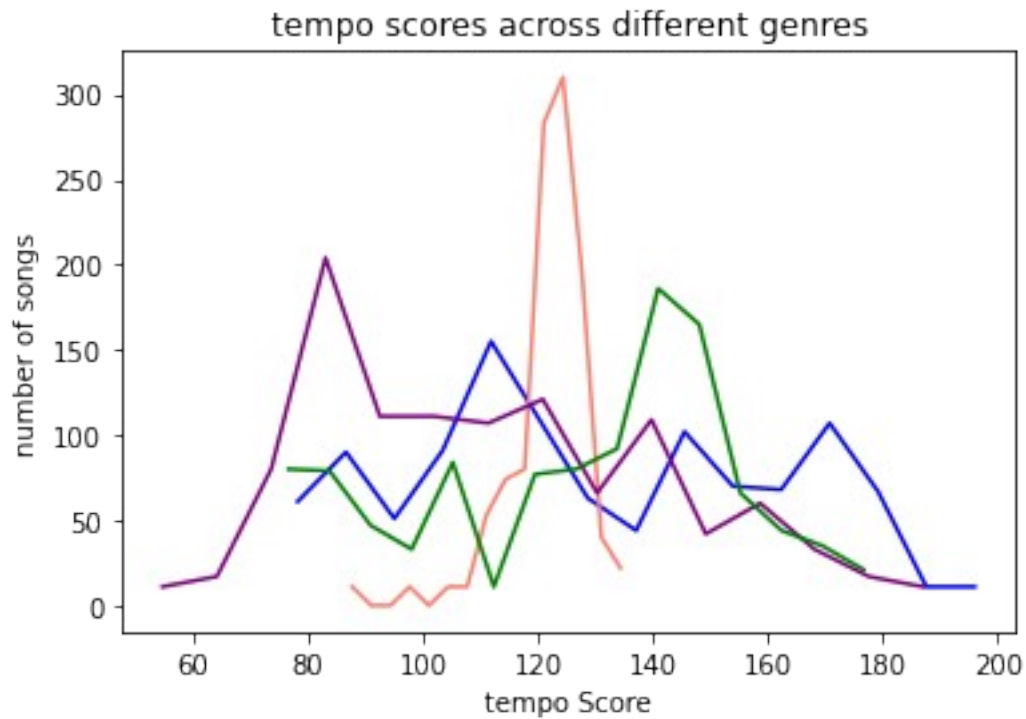
instrumentalness scores across different genres



valence scores across different genres







The above distributions reveal that for most features such as liveness, length, and acousticness, the shape of the distribution is mostly the same, and we can probably assume that these features do not have much variation across genres. However, some features such as tempo, valence, and energy have dramatic differences among the different genres, revealing that these

features might be pretty relevant to predicting genre. For example, it looks like for most party songs, there is a distinct tempo range around 120-130 that a majority of party songs fall into.

IV. BUILDING THE MACHINE LEARNING MODELS

For this part, we can test out some machine learning algorithms and see what works best for predicting genres for our specific type of dataset. The first step would be to import all the libraries we might use for training and testing our models.

```
from sklearn.preprocessing import LabelEncoder,MinMaxScaler
from sklearn.model_selection import train_test_split,
StratifiedKFold,KFold, cross_val_score
from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import categorical_crossentropy
from sklearn.tree import DecisionTreeClassifier
import pandas
from keras.utils import np_utils
```

Preparing the data

To prepare the data for training the models, we will take the raw data from the CSV and split it into inputs and output. The input will be all of the values corresponding to the audio features, and the output will be the playlist genre.

```
df = pandas.read_csv('data.csv') # Read the CSV made earlier into a
new dataframe

# Set an array of audio features as input
X = df[['popularity', 'danceability','acousticness', 'energy',
'loudness','tempo','valence']]
# Set the playlist genre as output
Y = df[df.columns[11:]]
```

Words do not mesh well with machine learning, so the outputs need to be encoded into numbers so that the algorithm can process it as a part of its calculations. The type of encoding used here is one-hot encoding, which essentially assigns each possible playlist genre as its own column and gives that column a 1 if it is the assigned genre (the rest get 0's). This avoids the problem of potentially making the algorithm think there is some type of ordinal relationship between our genres. For example, if rap, rock, and mellow were each 1, 2, and 3 respectively, it would not be good if the algorithm processed these as ranks, leading to false assumptions such as believing rock (2) was closer to rap (1) than mellow (3). Instead we want each genre to be treated equally without adding any false relations between them.

```
# Encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
```

```
encoded_Y = encoder.transform(np.ravel(Y))  
# Convert integers to dummy variables (i.e. one hot encoded)  
dummy_y = np_utils.to_categorical(encoded_Y)
```

In case there are features (such as popularity) that fall outside the range 0-1, we will scale the features to ensure they all fall between 0 and 1 while preserving the shape of the distribution using MinMaxScaler.

```
scaler = MinMaxScaler(feature_range = (0, 1)) # This scaler will scale  
the data in each column so that it falls between 0 and 1.  
X = scaler.fit_transform(X)
```

Finally, we can split our dataset into a training set and a testing set. This will allow for training on 80% of the data and leave the rest of the data as "unseen" by the model so that we have new data leftover to test it.

```
# Split the features into training and test data  
X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.4)
```

Model 1: Building a decision tree classifier model

The first model is going to be a decision tree classifier. This type of algorithm is best for large datasets (larger than the one currently being used), and ideal for simple relationships between features. It is quick to train, but it is prone to overfitting (when the predictions match to the train data too well and does not generalize to other data beyond the training set), so many parameters such as min_samples_leaf should be optimized to reduce overfitting.

```
# Create model  
decision_model =  
DecisionTreeClassifier(min_samples_leaf=5, min_impurity_decrease=0.001)  
  
# Fit data to model  
decision_model.fit(X_train, np.ravel(Y_train))  
# Run stratified kfold cross validation to evaluate the model  
skfold = StratifiedKFold(n_splits=10)  
cv_results = cross_val_score(decision_model, X_train,  
np.ravel(Y_train), cv=skfold, scoring="accuracy")  
msg = "The mean accuracy of the decision tree: %f (%f)" %  
(cv_results.mean(), cv_results.std())  
print(msg)
```

```
The mean accuracy of the decision tree: 0.955682 (0.016164)
```

Many parameters need to be optimized in order to produce valid results. One issue I noticed was that the training and validation set were very similar to each other because they are all based on the same playlist. I randomly selected some songs to use as "test" data to see how the machine learning model would work in the "real world", or completely random songs that are not related to the training data in any way. The following are some predictions in the random songs I picked

out. The best accuracy achieved was 0.69, but it is important to note the small sample size for the hand-picked test set.

#Testing random hand-picked URI's

```
sample_songs = [("Get You The Moon - Kina", "4ZLzo0kj0MPWrTLvooIuaa",
"mellow"), ("Pears - Weston Estate", "06nKF46jG8plzwyP4ziAyG",
"mellow"), ("Apocalypse - Cigarettes After Sex",
"3AVrVz5rK8Hrqo9YGiVGN5", "mellow"), ("Camelot - NLE Choppa",
"4sWuPniHClkInYAnzXAYUF", "rap"), ("Memories - David Guetta and Kid
Cudi", "4dTaiV9xFFCxnPur9c9yL", "party"), ("Give Me Everything -
Pitbull", "4QNpBfC0zvjKqPJcyqBy9W", "party"), ("Hot Shower - Chance the
Rapper", "00ko9WaS4j0X1kEk3gvHjf", "rap"), ("Chasing Gold - Dustin
Tebbutt", "2IOTVMVWfcLRdfRuYbMMnc", "mellow"), ("The Kids Aren't
Alright - Fall Out Boy", "3x7NGLfDKQYN2wnwEnL7r9", "rock"), ("Let It Die
- Three Days Grace", "6NAmbftWAovcMCnJzDCFsU", "rock"), ("While We're
Young - Jhene Aiko", "4mL59LVbKgOpEACxraGYdr", "mellow"), ("Crushing on
You - Finding Hope", "0FD064bveqjfNEIalB7HUr", "mellow"), ("Don't Play
That - King Von, 21 Savage", "3IPGG5Niex4hGlXF0YWzp9", "rap")]
X_test = []
Y_test = []
for (name, uri, genre) in sample_songs:
    print(name, "Genre: ", genre)
    x = features(uri, 1, token)
    x.pop()
    X_test.append(x)
    Y_test.append([genre])
    print("The predicted genre: ", gradient_model.predict([x]))
    print("")
```

```
Get You The Moon - Kina Genre:  mellow
The predicted genre:  ['rock']
```

```
Pears - Weston Estate Genre:  mellow
The predicted genre:  ['rock']
```

```
Apocalypse - Cigarettes After Sex Genre:  mellow
The predicted genre:  ['rap']
```

```
Camelot - NLE Choppa Genre:  rap
The predicted genre:  ['rap']
```

```
Memories - David Guetta and Kid Cudi Genre:  party
The predicted genre:  ['rock']
```

```
Give Me Everything - Pitbull Genre:  party
The predicted genre:  ['rock']
```

```
Hot Shower - Chance the Rapper Genre:  rap
The predicted genre:  ['rap']
```

Chasing Gold - Dustin Tebbutt Genre: mellow
The predicted genre: ['rap']

The Kids Aren't Alright - Fall Out Boy Genre: rock
The predicted genre: ['rock']

Let It Die - Three Days Grace Genre: rock
The predicted genre: ['rock']

While We're Young - Jhene Aiko Genre: mellow
The predicted genre: ['rock']

Crushing on You - Finding Hope Genre: mellow
The predicted genre: ['rock']

Don't Play That - King Von, 21 Savage Genre: rap
The predicted genre: ['rap']

Fine tuning hyperparameters in machine learning is a complicated and lengthy process. In order to partially fine tune the model, I picked three parameters to focus on and retrained the model on various combinations of the parameters. Its possible to tune it much more than in this example if more parameters and more complicated tuning functions like GridSearch were used.

```
min_samples_leaf = [2,3,4,5,10,15]
min_impurity_decrease = [0.00001, 0.00005, 0.0001, 0.0005, 0.01, 0.10,
1]
max = 0
params = []
print("Running...")
for leaf in min_samples_leaf:
    for impurity in min_impurity_decrease:
        for depth in max_depth:
            decision_model =
DecisionTreeClassifier(min_samples_leaf=leaf,min_impurity_decrease=imp
urity)
            decision_model.fit(X_train, np.ravel(Y_train))
            cv_results = cross_val_score(decision_model, X_train,
np.ravel(Y_train), cv=skfold, scoring="accuracy")
            y_pred = decision_model.predict(X_test)
            if accuracy_score(y_pred, Y_test)>max:
                max = accuracy_score(y_pred, Y_test)
                params = [estimator, feature, depth]
print("Ideal parameters for best accuracy: ",params, " Best accuracy
acheived: ",max)

Running...
Ideal parameters for best accuracy: [50, 7, 10] Best accuracy
acheived: 0.6153846153846154
```

Model 2: Building a gradient boosting classifier

A gradient boosting classifier works by sequentially adding predictors to the ensemble as it creates many weak models based on previous models, and this repeated training eventually results in better accuracy. The problem with this type of classifier is that it is very hard to fine tune due to the many parameters and potentially range of values it can take up.

```
from sklearn.ensemble import GradientBoostingClassifier

gradient_model = GradientBoostingClassifier(n_estimators=5,
learning_rate=1, max_features=6, max_depth=3, random_state=0)
gradient_model.fit(X_train, np.ravel(Y_train))

skfold = StratifiedKFold(n_splits=10)
cv_results = cross_val_score(gradient_model, X_train,
np.ravel(Y_train), cv=skfold, scoring="accuracy")
msg = "The mean accuracy of the decision tree: %f (%f)" %
(cv_results.mean(), cv_results.std())
print(msg)
```

The mean accuracy of the decision tree: 0.959470 (0.014675)

The new gradient model is tested on the same dataset as above to see the differences in predictions. Then just like for the previous model, we will tune some parameters and try to maximize how many of the test data we can accurately predict.

```
#Testing random hand-picked URI's
sample_songs = [("Get You The Moon - Kina", "4ZLzo0kj0MPWrTLvooIuaa",
"mellow"), ("Pears - Weston Estate", "06nKF46jG8plzwyP4ziAyG",
"mellow"), ("Apocalypse - Cigarettes After Sex",
"3AVrVz5rK8Hrqo9YGiVGN5", "mellow"), ("Camelot - NLE Choppa",
"4sWuPniHClkInYAnzXAYUF", "rap"), ("Memories - David Guetta and Kid
Cudi", "4dTaiV9xFFCxnPur9c9yL", "party"), ("Give Me Everything -
Pitbull", "4QNpBfC0zvjkQpJcyqBy9W", "party"), ("Hot Shower - Chance the
Rapper", "00ko9WaS4j0X1kEk3gvHjf", "rap"), ("Chasing Gold - Dustin
Tebbutt", "2IOTVMVWfclRdfRuYbMMnc", "mellow"), ("The Kids Aren't
Alright - Fall Out Boy", "3x7NGlfdKQYN2wnwEnL7r9", "rock"), ("Let It Die
- Three Days Grace", "6NAmbftWAovcMCnJzDCFsU", "rock"), ("While We're
Young - Jhene Aiko", "4mL59LVbKgOpEACxraGYdr", "mellow"), ("Crushing on
You - Finding Hope", "0FD064bveqjfNEIalB7HUr", "mellow"), ("Don't Play
That - King Von, 21 Savage", "3IPGG5Niex4hGlXF0YWzp9", "rap")]
X_test = []
Y_test = []
for (name, uri, genre) in sample_songs:
    print(name, "Genre: ", genre)
    x = features(uri, 1, token)
    x.pop()
    X_test.append(x)
    Y_test.append([genre])
```

```
print("The predicted genre: ",gradient_model.predict([x]))  
print("")
```

Get You The Moon - Kina Genre: mellow
The predicted genre: ['rap']

Pears - Weston Estate Genre: mellow
The predicted genre: ['rock']

Apocalypse - Cigarettes After Sex Genre: mellow
The predicted genre: ['rap']

Camelot - NLE Choppa Genre: rap
The predicted genre: ['rock']

Memories - David Guetta and Kid Cudi Genre: party
The predicted genre: ['rap']

Give Me Everything - Pitbull Genre: party
The predicted genre: ['rap']

Hot Shower - Chance the Rapper Genre: rap
The predicted genre: ['rap']

Chasing Gold - Dustin Tebbutt Genre: mellow
The predicted genre: ['mellow']

The Kids Aren't Alright - Fall Out Boy Genre: rock
The predicted genre: ['rap']

Let It Die - Three Days Grace Genre: rock
The predicted genre: ['rap']

While We're Young - Jhene Aiko Genre: mellow
The predicted genre: ['rock']

Crushing on You - Finding Hope Genre: mellow
The predicted genre: ['rap']

Don't Play That - King Von, 21 Savage Genre: rap
The predicted genre: ['rap']

Finally, we can try to tune the model a little so that it can predict the above values a bit more accurately. Running the following code, the max accuracy achieved was 0.84. This can be further increased by exploring more parameters to tune and exploring a wider range of values for different parameters.

```

n_estimators = [5,10,15,20,30,50]
max_features= [2,3,4,5,6,7]
max_depth=[2,5,10,15,20,50]
max = 0
params = []
print("Running...")
for estimator in n_estimators:
    for feature in max_features:
        for depth in max_depth:
            gradient_model =
GradientBoostingClassifier(n_estimators=estimator, learning_rate=1,
max_features=feature, max_depth=depth)
            gradient_model.fit(X_train, np.ravel(Y_train))
            cv_results = cross_val_score(decision_model, X_train,
np.ravel(Y_train), cv=skfold, scoring="accuracy")
            y_pred = gradient_model.predict(X_test)
            if accuracy_score(y_pred, Y_test)>max:
                max = accuracy_score(y_pred, Y_test)
                params = [estimator, feature, depth]
print("Ideal parameters for best accuracy: ",params, " Best accuracy
acheived: ",max)

Running...
Ideal parameters for best accuracy: [10, 3, 20] Best accuracy
acheived: 0.7692307692307693

```

V. Discussion

There are many possible ways to improve the above models to increase their accuracies. However, based on the current attempts to hypertune each both models, it looks like the Gradient boosting algorithm seems to do better for predicting genres for test data songs that the model is unfamiliar with.

Both models had a very high accuracy when trained, but when tested on the small test sample that I created, the accuracy is very different (lower). This suggests that the models have both overfitted quite a bit to the training data. This means that the models calculations are very fitted to the training dataset and any new songs that are not relevant to the training data would have poorer predictions. The gradient boosting algorithm seems to be a lot better with unfamiliar data since we were able to achieve an accuracy of 0.84 on the testing set and 0.96 on the training set. The decision tree is overfitted because the training set has a high accuracy of 0.94 but a much lower testing accuracy of 0.69. This would likely need more fine tuning to reduce overfitting and improve predictions.

Additionally, none of the predictions seem to be able to predict party songs well. This is likely because party song features overlaps greatly with rap (the prominent beat), and rock (the high energy). Therefore many party songs were instead classified as rap or rock.

The training dataset was made by repeated adding Spotify's recommended music that was similar to songs in the current playlist. Although this is a convenient way to increase volume of the dataset, it reduces the diversity in the data because it is all songs that Spotify considers

similar, and thus the range of artists/sounds would be smaller. In the future, it may be better to diversify the training dataset to source from multiple different playlists related to a genre so that the models can generalize better to unfamiliar testing data since it will have trained on a greater variety and volume of data. This can also help with the overfitting problem because overfitting to a more diverse dataset can help it generalize better.

The other main issue is overfitting, and this would need to be solved through a series of lengthy and complex parameter selection and tuning. I attempted a bit of tuning in the two models above, but this fine tuning would need to be more much in depth in order to properly optimize the parameters. I only selected a couple parameters that are commonly tuned and used a limited range of values to test them out on.

Lastly, better feature selection can improve predictions. Currently, only some audio features are used as a factor in the prediction. Adding other features such as artist, lyrical analysis (such as happiness or repetitiveness of lyrics) can refine the predictions and potentially be able to differentiate party songs from the rest of the genres.

VI. Conclusion

Spotify uses much more than just audio features to classify and recommend songs. A major aspect of it is collaborative filtering which uses data from other user's past interactions with songs to make sense of how another user might perceive a song. As an oversimplified example, someone who has a similar music taste to a given user likes a song, this song will likely be recommended to the given user. So mimicking Spotify's method of genre classification would be a very complicated process that would have to involve much more than just a few audio features.

This project was an exploratory project to try out machine learning for the first time on a topic that greatly interested me. I spend hours creating playlists and listening to music, so trying to tie machine learning into this was a huge challenge that revealed a lot about the complicated process of data science. Creating the dataset and hypertuning the models were the two steps that took a majority of the time because there are many possible ways to organize a dataset and choose features to use, and there are also many ways to tune a model to try to improve the accuracy. It is a very iterative process that requires a lot of trial and error in order to improve predictions.